torch.autograd.grad#

https://pytorch.org/docs/stable/generated/torch.autograd.grad.html#torch.aut

Description:

Compute and return the sum of gradients of outputs with respect to the inputs. grad_outputs should be a sequence of length matching output containing the “vector” in vector-Jacobian product, usually the pre-computed gradients w.r.t. each of the outputs. If an output doesn’t require_grad, then the gradient can be None). If you run any forward ops, create grad_outputs, and/or call grad in a user-specified CUDA stream context, see Stream semantics of backward passes.

Function Signature

```
torch.autograd.grad(outputs, inputs, grad_outputs=None,
retain_graph=None, create_graph=False, only_inputs=True,
allow_unused=None, is_grads_batched=False,
materialize_grads=False)[source]#

Parameters

Return type
```

Returns:

tuple[torch.Tensor, …]

Notes & Warnings

NOTE Note If you run any forward ops, create grad_outputs, and/or call grad in a user-specified CUDA stream context, see Stream semantics of backward passes.

NOTE Note only_inputs argument is deprecated and is ignored now (defaults to True). To accumulate gradient for other parts of the graph, please use torch.autograd.backward.

Detailed Documentation

Documentation

Compute and return the sum of gradients of outputs with respect to the inputs.

grad_outputs should be a sequence of length matching output containing the “vector” in vector-Jacobian product, usually the pre-computed gradients w.r.t. each of the outputs. If an output doesn’t require_grad, then the gradient can be None).

If you run any forward ops, create grad_outputs, and/or call grad in a user-specified CUDA stream context, see Stream semantics of backward passes.

only_inputs argument is deprecated and is ignored now (defaults to True). To accumulate gradient for other parts of the graph, please use torch.autograd.backward.

outputs (sequence of Tensor or GradientEdge) – outputs of the differentiated function.

inputs (sequence of Tensor or GradientEdge) – Inputs w.r.t. which the gradient will be returned (and not accumulated into .grad).

grad_outputs (sequence of Tensor) – The “vector” in the vector-Jacobian product. Usually gradients w.r.t. each output. None values can be specified for scalar Tensors or ones that don’t require grad. If a None value would be acceptable for all grad_tensors, then this argument is optional. Default: None.

retain_graph (bool, optional) – If False, the graph used to compute the grad will be freed. Note that in nearly all cases setting this option to True is not needed and often can be worked around in a much more efficient way. Defaults to the value of create_graph.

create_graph (bool, optional) – If True, graph of the derivative will be constructed, allowing to compute higher order derivative products. Default: False.

allow_unused (Optional[bool], optional) – If False, specifying inputs that were not used when computing outputs (and therefore their grad is always zero) is an error. Defaults to the value of materialize_grads.

is_grads_batched (bool, optional) – If True, the first dimension of each tensor in grad_outputs will be interpreted as the batch dimension. Instead of computing a single vector-Jacobian product, we compute a batch of vector-Jacobian products for each “vector” in the batch. We use the vmap prototype feature as the backend to vectorize

calls to the autograd engine so that this computation can be performed in a single call. This should lead to performance improvements when compared to manually looping and performing backward multiple times. Note that due to this feature being experimental, there may be performance cliffs. Please use torch._C._debug_only_display_vmap_fallback_warnings(True) to show any performance warnings and file an issue on github if warnings exist for your use case. Defaults to False.

materialize_grads (bool, optional) – If True, set the gradient for unused inputs to zero instead of None. This is useful when computing higher-order derivatives. If materialize_grads is True and allow_unused is False, an error will be raised. Defaults to False.

tuple[torch.Tensor, …]
